SpecChain: Part 5 – Auditing Governance and Incentive Networks

SpecChain Working Group

September 17, 2025

Abstract

This part consolidates and refines SpecChain's auditing and governance protocols, ensuring consistency with the formal semantics of Parts 1–4. We specify runtime audit rules, evidence bundling, and incentive-related constructs under the fixed execution pipeline EF→DRO→PR→CBF→ Ω. Terminology and algebraic conventions (acceptance algebra Q, obligations Σ, unknowns $\perp$, EnvLock) follow SCp1–4. We normalize or footnote any speculative mechanisms from prior drafts, present canonical JSON schemas for receipts and certificates, and summarize the guard checks and audit emissions in a reference table. All JSON formats use `canon/v1` encoding with explicit witness and digest fields.

1 Preliminaries and Notation

We inherit the algebraic and loader semantics of Parts 1–4 and make the following *normative* choices for Part 5.

N0. Canonical bytes and digests. All evidence and receipts are encoded as canonical JSON `canon/v1` (sorted keys, minimal decimals, UTF-8). *Every* digest in this Part uses BLAKE3-256 and is rendered as a string `"b3:..."`. The run fingerprint field is `proof_surface`. Any legacy artifact that used a different name or hash (e.g., `proof_print`, SHA-256) *must* be rebound with a LegacyRebindCert that records both the legacy digest and its BLAKE3 binding (see §**??**).

N1. Determinism envelope. All execution and replay occur under a pinned EnvLock/ReplayPlan: fixed FP modes, PRNG family (if any), seeds, library hashes, and canonical traversal orders. No ambient entropy is consulted. Byte-identical replay from the evidence bundle is required.

N2. Acceptance algebra. Acceptance states $q \in Q$ are a typed product of axes partitioned into sum-axes S (cumulative) and max-axes M (idempotent). We adopt the aliases

$$\odot \equiv \uplus \quad \text{(horizontal aggregation; sum on } S\text{, max on } M\text{)}, \qquad \wedge \equiv \star \quad \text{(vertical sequencing)}.$$

Unknowns use the absorbing bottom $\perp$. Once ledgered, $\perp$ is irrevocable on its axis and propagates under both $\odot$ and $\wedge$.

N3. Guard-first rule (fail-closed). No priced contribution may pass a guard if any of its priced inputs are $\perp$. Such attempts *must* be rejected before entering Q and emit a RejectionCert (reason code `C_UNK_PRICED`).

N4. Axis isolation. Each adapter declares a whitelist of axes it may write. Any undeclared write is a violation (`C_AXIS_LEAK`) and halts with a RejectionCert. Isolation is verified per-stage.

N5. Policy-kernel abstraction. Domain logic (guards, pricing, incentives) is implemented as *policy-pinned kernels* over *normalized, dimensionless* inputs. Kernels are pure functions identified by `checker_hash` and consume only canonical input views. Thresholds/salts are policy, not code constants.

N6. Receipts (uniform shape). Every predicate emits a canonical receipt with fields `predicate_id`, `inputs_digest`, `checker_hash`, `outcome.verdict`$\in$ {`PASS`, `FAIL`}, and on failure a deterministic `outcome.reason_code`. All receipts are referenced by digest inside the bundle manifest.

N7. Compliance Bundle (top-level container). Each window produces a *Compliance Bundle* whose bytes are bound into `proof_surface`. Required keys (all content-addressed BLAKE3 digests unless the key is a list):

- `attestation_receipt` (AttestationReceipt),
- `guard_receipts` (ordered EF, DRO, PR, CBF),
- `omega_receipt` (§??),
- `bundle_manifest` (Merkle root + indices of all artifacts),
- `axis_catalog_digest`, `policy_digests`.

Execution order. The Loader's evaluation pipeline is acyclic and fixed:

$$\text{EF} \;\rightarrow\; \text{DRO} \;\rightarrow\; \text{PR} \;\rightarrow\; \text{CBF} \;\rightarrow\; \Omega.$$

At each stage a guard is evaluated under EnvLock, emitting a PASS/FAIL receipt bound into the evidence bundle. The terminal stage Ω applies the horizon gate (§??) and closes the bundle.

Budgets and obligations. At Ω, the Loader compares the aggregate q to the policy budget vector B axis-wise and requires $q \leq B$ for acceptance. Obligations Σ track pending requirements and are ordered lexicographically by a rank μ; any step that discharges an obligation strictly decreases μ, ensuring no-Zeno progress. Update paths (UpdateCert) must satisfy step-preserving bisimulation and Σ-non-expansion.

Hash discipline (legacy interop). If any artifact employs non-BLAKE3 digests or the legacy `proof_print` name, a LegacyRebindCert *must* be included to bind the legacy bytes to BLAKE3 and to `proof_surface`. New artifacts *must not* introduce additional hash families.

Notation summary. We use $\perp$ for unknowns, $\odot/\wedge$ as above, S/M for axis classes, and reserve `b3:...` for BLAKE3-256 strings. JSON examples throughout are illustrative but conform to `canon/v1` serialization rules.

2 Runtime Guards and Acceptance Checks

This section specifies the Loader's guard sequence and the uniform evidence it emits. Each guard is a *policy-pinned kernel* over a canonical, dimensionless input view (§1, N5). Evaluation occurs under EnvLock/ReplayPlan (N1). Every guard produces a uniform receipt (N6) whose digest is included in the window's Compliance Bundle (N7) and thus in `proof_surface`.

G0. Kernel contract (normative). A guard kernel K is a pure function

$$K : (\texttt{canon/v1 inputs}, \texttt{policy}) \longrightarrow \{\texttt{PASS}, \texttt{FAIL}\}$$

with:

1. *Canonical input view* x: a JSON object of normalized, *dimensionless* fields and an explicit `salt`.
2. *Checker identity* `checker_hash`: BLAKE3-256 of the kernel implementation bytes.
3. *Receipt*: `predicate_id`, `inputs_digest` $=$ b3(bytes(x)), `checker_hash`, `outcome.verdict` $\in \{\texttt{PASS}, \texttt{FAIL}\}$, and on failure `outcome.reason_code` (deterministic).

Guards *must* enforce (N3) Guard-first: any priced input equal to $\perp$ yields `FAIL` with `C_UNK_PRICED`.

Evaluation order. The Loader evaluates guards in the fixed order

$$\text{EF} \to \text{DRO} \to \text{PR} \to \text{CBF} \to \Omega,$$

halting on first failure with a RejectionCert. The terminal Ω gate is defined in §?? and emits an *OmegaReceipt* (§??).

EF guard (example input view). An EF kernel consumes a canonical view such as:

Listing 1: EF canonical input view (illustrative)

```
{
  "normalized_signal": 0.82,
  "normalized_context_radius": 0.47,
  "salt": "ef-locality-v1"
}
```

The kernel's predicate and any clamp mapping are policy parameters; only the receipt fields listed in Table 1 are normative.

DRO guard and quarantine. On `FAIL`, the DRO kernel must populate `outcome.reason_code="C_DRO_SIDE"` and the Loader emits a RejectionCert with a minimal counterexample witness (N6). A *Quarantine* obligation is added to Σ and must be discharged via subsequent evidence per policy; obligations cannot be erased without a certified update (Σ-non-expansion).

PR profiles and retune evidence. The PR stage may *accept* a RetuneCert/UpdateCert reference as a witness but may not write priced axes. Any retune effect applied to model parameters is evidenced solely via the receipt's `witness_digests`; adapter outputs remain axis-isolated (N4).

CBF crossings and ISS invariants. Each admissible crossing is executed as a canonical *CrossingPacket* with fields `edge_id`, `inputs_digest`, `kernel_id@hash`, `priced_delta` (typed Q-vector), `sequence_no`, `prev_hash`, and `packet_hash`. The Loader verifies:

1. **Idempotence on replay**: duplicate (`edge_id`, `sequence_no`) re-executes to byte-identical `priced_delta`, else `C_ORD_DRIFT`.

Table 1: Guard sequence and emitted evidence (all digests BLAKE3-256 as `b3:...`).

Stage (predicate_id)	**Canonical check (policy kernel)**	**Emitted evidence (receipt keys)**
EF (`EF_LOCALITY`)	Evaluate locality/visibility constraints on normalized features; set a deterministic clamp label (e.g., `GREEN/AMBER/RED`) as *metadata only*. Guard passes iff the policy predicate over x is true.	`predicate_id`, `inputs_digest`, `checker_hash`, `outcome.verdict`, metadata `clamp`; digest → `guard_receipts[0]`.
DRO (`DRO_SIDECHANNEL`)	Check side-channel consistency on normalized slope/variance features; on failure, set a *Quarantine* obligation in Σ.	Receipt as above; if FAIL: RejectionCert with `C_DRO_SIDE` and obligation emit.
PR (`PR_PROFILE`)	Verify profile margins and any offline retune witness; the kernel may *read* retune certificates but cannot modify priced axes.	Receipt; optional linkage to applied retune proof digests in `witness_digests`.
CBF (`CBF_LEDGER_INTEGRITY`)	Apply admissible crossings as *Crossing-Receipts* and verify ISS ledger invariants (idempotence on replay; conservation over the window).	`CBF_AUDIT` receipt: packet sequence digest, initial/final ISS digests, verdict; per-packet digests bound via `bundle_manifest`.
Ω (`OMEGA_HORIZON`)	Compute and check the horizon gate K from $(\tau, N, \tau_{\min})$; ensure $q \le B$ axis-wise for final acceptance.	`OmegaReceipt`: `inputs_digest` of $(\tau, N, \tau_{\min})$, `computed_k`, verdict; terminal ReplayReceipt binds `proof_surface`.

2. **Ledger conservation**: the window's ISS delta equals the $\odot$-fold of `priced_deltas` in canonical order; else `C_LEDGER_DRIFT`.

These checks are summarized in a `CBF_AUDIT` receipt whose digest is included in `guard_receipts` and the bundle manifest.

Terminal checks and acceptance. The Ω stage performs the horizon gate (formalized in §??) and the final budget comparison $q \le B$. Acceptance requires *all* guards PASS plus $q \le B$; otherwise the Loader halts with a RejectionCert. The terminal ReplayReceipt contains the bundle Merkle root and the final `proof_surface` digest.

Failure codes (this section).

Code	**Origin**	**Meaning**
`C_UNK_PRICED`	Guard-first	Priced input $\perp$ encountered; pre-aggregation rejection
`C_AXIS_LEAK`	Axis isolation	Adapter wrote undeclared axis
`C_DRO_SIDE`	DRO guard	Side-channel predicate failed; quarantine set
`C_LEDGER_DRIFT`	CBF audit	Conservation/idempotence violation in ISS
`C_HZN_VIOL`	Ω gate	Horizon mismatch or recomputation failure

Each failure emits a uniform receipt (N6) and a minimal RejectionCert witness.

3 Migration and Legacy Interoperability

This appendix specifies a *normative* path for upgrading legacy artifacts (e.g., `proof_print`, SHA-256 digests, ad-hoc transport packs) into the Part 5 regime (BLAKE3-only digests, `proof_surface`, Compliance Bundle).

3.1 Scope and Goals

M0. Goals. (1) Preserve audit soundness for finalized legacy runs; (2) produce a single, canonical fingerprint `proof_surface` for the upgraded bundle; (3) avoid any change to historical ACCEPT/HALT verdicts (bisimulation).

Artifacts considered *legacy* include: (i) runs whose terminal fingerprint field is named `proof_print`; (ii) receipts or packs whose digests are not BLAKE3-256; (iii) "transport pack" formats that do not match the Compliance Bundle schema (§??).

3.2 Mandatory Rebinding Workflow

Upgrading a single legacy window produces a new Compliance Bundle and a LegacyRebindCert that binds old and new identifiers.

M1. Byte-stable reconstruction.

1. **Canonicalization:** Re-encode all receipts/certificates as `canon/v1` JSON (sorted keys, minimal decimals).
2. **Digest lift:** For every legacy digest d_{old}, compute $d_{\text{b3}} = \text{BLAKE3}(\text{bytes})$. Retain both in the upgraded receipt as `legacy_digest` and `digest`.
3. **Bundle assembly:** Construct the Compliance Bundle keys (`attestation_receipt`, `guard_receipts`, `omega_receipt`, `bundle_manifest`, `axis_catalog_digest`, `policy_digests`).
4. **Surface binding:** Compute `proof_surface` over the upgraded bundle bytes; issue a LegacyRebindCert linking the legacy terminal fingerprint (if any) to `proof_surface`.

Listing 2: LegacyRebindCert for a single window

LegacyRebindCert (normative fields).

```
{
  "kind": "LegacyRebindCert",
  "legacy_proof_print": "sha256:old-surface-...",
  "proof_surface": "b3:new-surface-...",
  "canon": "v1",
  "time": 0,
  "stage": "Omega",
  "outcome": { "verdict": "PASS" }
}
```

3.3 Acceptance Rules for Mixed Bundles

When a Compliance Bundle references artifacts that still carry legacy digests, the following additional rules apply.

M2. Dual-binding discipline. Any field that retains a legacy digest *must* also include a `digest` field with the BLAKE3 value over the same bytes. Verifiers ignore legacy digests for acceptance; they serve only for cross-reference.

M3. Bisimulation invariance. The upgrader must provide a *GovernanceReceipt*-style witness (see **??**) or a dedicated `REPLAY_IMPACT` receipt that demonstrates the ACCEPT/HALT bit for the legacy run is unchanged under the upgraded bundle. Violation ⇒ `C_GOV_NO_CERT`.

3.4 Deprecation Timeline (Policy)

M4. Phases.

1. *Phase I (coexistence):* Dual-binding permitted; LegacyRebindCert mandatory when legacy fields appear.
2. *Phase II (strict):* No new artifacts may carry legacy digests or `proof_print`; existing archives must be accompanied by rebinding certs.
3. *Phase III (enforcement):* Validators *reject* bundles that contain legacy-only digests or lack required rebinding (`C_ATT_MISSING`/`C_ORD_DRIFT`).

Policy sets concrete dates (axis-catalog entry) and is itself changed only via UpdateCert.

3.5 Upgrade Checklist (Auditor-Oriented)

M5. Minimal checks. (1) All receipts canonicalized (`canon/v1`); (2) BLAKE3 digests present for every artifact referenced by the manifest; (3) LegacyRebindCert included if any legacy field remains; (4) `proof_surface` recomputed from the upgraded bundle; (5) Omega K recomputed byte-identically (§**??**); (6) CBF conservation/idempotence verified for any reconstructed *CrossingPacket* list.

3.6 Worked Example (Sketch)

Listing 3: Upgraded Compliance Bundle (abridged)

```
{
  "attestation_receipt": "b3:att-...",
  "guard_receipts": ["b3:ef-...", "b3:dro-...", "b3:pr-...", "b3:cbf-..."
     ],
  "omega_receipt": "b3:omg-...",
  "bundle_manifest": "b3:mrk-...",
  "axis_catalog_digest": "b3:axis-...",
  "policy_digests": ["b3:budgets-...", "b3:rounding-..."],
  "_compat": {
    "legacy": true,
```

```
    "legacy_proof_print": "sha256:old-surface-...",
    "rebind_cert": "b3:rebind-..."
  }
}
```

What migration guarantees. Upgrading yields a single `proof_surface` for each legacy window, preserves historical verdicts (bisimulation), and eliminates mixed-hash ambiguity via mandatory dual-binding and LegacyRebindCert. Auditors can replay upgraded runs byte-for-byte under EnvLock/ReplayPlan.

4 Conformance Suite and Reference Checks

This section provides a minimal, *normative* conformance suite—reference algorithms and test vectors—to mechanically validate implementations against Part 5. All procedures operate on `canon/v1` bytes and BLAKE3 digests (N0), under EnvLock/ReplayPlan determinism.

4.1 Predicate Registry (IDs and Stages)

Implementations *must* use the following canonical `predicate_id` strings per stage. These identifiers appear in receipts (N6) and are input to allow-lists, dashboards, and CI checks.

Table 2: Canonical predicate registry (Part 5).

Stage	predicate_id	Purpose
Boot	`ATT_TCB`	TCB attestation
EF	`EF_LOCALITY`	Locality/visibility guard
DRO	`DRO_SIDECHANNEL`	Side-channel consistency guard
PR	`PR_PROFILE`	Profile/retune guard
CBF	`CBF_LEDGER_INTEGRITY`	Idempotence & conservation audit
Omega	`OMEGA_HORIZON`	Horizon K check
Load	`GOV_UPDATE_APPLIED`	Governance update accepted
CBF	`INCENTIVE_DEBIT`	Incentive debit/credit (per packet)
EF	`SANDBOX_NONACTUATION`	Sandbox isolation / ethics gate

4.2 Reference: Horizon Checker (Ω)

The following reference algorithm is *normative* for recomputing K from `inputs_digest` (see **??**). All numeric widths and rounding modes are pinned by ReplayPlan.

Listing 4: Reference pseudo-algorithm: Omega horizon checker

```
{
  "fn": "omega_check",
  "inputs": { "tau_ticks": "u64", "tau_min_ticks": "u64", "N": "u64" },
  "steps": [
    "require(tau_min_ticks > 0)",
    "let lhs = 2 * tau_ticks",
```

```
    "let q = ceil_div(lhs, tau_min_ticks)",     // ceiling division in EnvLock mode
    "let K = max(N, q)",
    "require(tau_ticks >= 2 * tau_min_ticks)", // horizon precondition
    "return K"
  ]
}
```

Test vectors (illustrative).

τ	$\tau_{\min}$	N	K (expected)	Verdict
100	40	20	$\max(20, \lceil 200/40 \rceil = 5) = 20$	PASS
100	51	60	$\max(60, \lceil 200/51 \rceil = 4) = 60$	**FAIL** ($\tau < 2\tau_{\min}$)
256	64	128	$\max(128, \lceil 512/64 \rceil = 8) = 128$	PASS

4.3 Reference: Bundle Manifest Root

Given an ordered list of leaves $\ell_i = \text{bytes}(\texttt{kind}, \texttt{index}, \texttt{digest})$ serialized in `canon/v1`, compute the Merkle root as follows:

Listing 5: Manifest Merkle root (reference)

```
{
  "fn": "manifest_root",
  "inputs": [ "leaf_bytes[0..n)" ],
  "steps": [
    "for all i: h[i] = BLAKE3(leaf_bytes[i])",
    "while len(h) > 1:",
    "  if odd(len(h)): append(h[-1])        // last hash duplicated (evening rule)",
    "  h = [ BLAKE3(h[2j] || h[2j+1]) for j in 0..len(h)/2 ) ]",
    "return h[0]"
  ],
  "notes": "Leaves are ordered lexicographically by (kind, index)."
}
```

Conformance checks. (1) Every digest referenced in the Compliance Bundle must appear as a leaf; (2) no duplicate `(kind,index)` pairs; (3) root must match `bundle_manifest`.

4.4 Reference: CBF Ledger Invariants

Listing 6: CBF invariants (idempotence & conservation)

```
{
  "fn": "cbf_audit",
  "inputs": {
    "packets": "[CrossingPacket] (ordered)",
    "iss_start": "Q-vector",
    "iss_end": "Q-vector"
  },
```

```
  "steps": [
    "// Idempotence on (edge_id, sequence_no)",
    "require(all_equal_delta_on_duplicates(packets))",
    "// Conservation",
    "let fold = q_identity()",
    "for p in packets: fold = q_hjoin(fold, p.priced_delta)",   //
       over S/M
    "require(q_equal(q_hjoin(iss_start, fold), iss_end))",
    "return PASS"
  ]
}
```

Duplicate check. For every fixed `(edge_id, sequence_no)`, recompute `priced_delta` from `inputs_digest` with the pinned kernel (`kernel_id@hash`) and byte-compare to the packet's recorded value.

4.5 Reference: Uniform Receipt Validator

Listing 7: Receipt schema validator (reference)

```
{
  "fn": "validate_receipt",
  "inputs": "receipt_json",
  "steps": [
    "require(has_keys(predicate_id, inputs_digest, checker_hash, outcome))
       ",
    "require(outcome.verdict in {PASS, FAIL})",
    "if outcome.verdict == FAIL: require(has_key(outcome.reason_code))",
    "require(stage in {Boot, Load, EF, DRO, PR, CBF, Omega})",
    "require(is_b3(inputs_digest) && is_b3(checker_hash))",
    "return PASS"
  ]
}
```

4.6 Determinism Probes

To detect non-determinism early, vendors should pass the following probes under EnvLock/ReplayPlan. A failure yields `C_INC_NONDET` (for incentives) or `C_LEDGER_DRIFT`/`C_ORD_DRIFT` otherwise.

- **Replay-twice probe:** Execute the same window twice from identical inputs; require byte-identical receipts and `proof_surface`.
- **Packet-shuffle probe:** Shuffle *equal-sequence* duplicates; require idempotence acceptance to hold.
- **RNG seal probe:** Verify that any kernel that formerly used RNG produces identical outputs when run with seeds derived solely from digests and salts.

4.7 Minimal Test Bundle (Abridged)

The following abridged Compliance Bundle demonstrates a PASS run with one crossing and a satisfied horizon gate. *Digests are placeholders.*

Listing 8: Abridged PASS bundle (placeholders)

```
{
  "attestation_receipt": "b3:att-1111",
  "guard_receipts": ["b3:ef-2222","b3:dro-3333","b3:pr-4444","b3:cbf-5555"
     ],
  "omega_receipt": "b3:omg-6666",
  "bundle_manifest": "b3:mrk-7777",
  "axis_catalog_digest": "b3:axis-8888",
  "policy_digests": ["b3:bud-9999","b3:eth-aaaa"]
}
```

4.8 Conformance Checklist (CI)

CI steps. (1) Schema-validate all receipts (**??** 7); (2) recompute K and check `OmegaReceipt` (**??** 4); (3) reconstruct manifest root (**??** 5); (4) run CBF audit (**??** 6); (5) verify axis isolation and Guard-first; (6) confirm ReplayReceipt & `proof_surface` recomputation; (7) if updates present, validate `GOV_UPDATE_APPLIED` with quorum and proofs.

Outcome. Passing the conformance suite guarantees byte-replayability, correct horizon gating, ledger conservation, receipt uniformity, and governance safety—constituting a minimum for interoperability across Part 5 implementations.

5 Security Considerations and Threat Model

This section summarizes the adversary model, key attack surfaces, and how Part 5's normative checks (§2–4) mitigate them. All guarantees are enforced under EnvLock/ReplayPlan and are evidenced by canonical receipts bound into `proof_surface`.

5.1 Adversary Model

- **A1 (Byzantine producer).** May emit malformed receipts, reorder or omit artifacts, or try to mutate policy out-of-band.
- **A2 (Non-determinism injector).** Attempts to use RNG, FP drift, platform variance, or time to create unreplayable outputs.
- **A3 (Ledger manipulator).** Attempts duplicate or conflicting *CrossingPackets*, or to violate conservation/idempotence.
- **A4 (Governance capture).** Tries to change budgets/axes without quorum or without formal safety proofs (bisimulation, Σ-non-expansion).
- **A5 (Ambiguity attacker).** Exploits mixed hash families or legacy names to create digest confusion/replay ambiguity.
- **A6 (Axis bleed).** Writes to undeclared axes to launder priced effects, or launders unknowns $\perp$ past guards.
- **A7 (Incentive feedback).** Spends on incentive axes to loosen guards in-window; introduces non-deterministic debits.

- **A8 (Sandbox escape).** Writes to priced axes or simulates ethically forbidden scenarios under the guise of "what-if."

5.2 Threats → Invariants → Mitigations

Table 4: Threat matrix (codes per §??).

Threat	Invariant / Receipt	Mitigation / Failure
Evidence forgery / omission (A1)	`ATT_TCB`, `ORD_PIPE`, manifest invariants (CB2)	All referenced digests must appear as manifest leaves; stage order EF→DRO→PR→CBF→ Ω enforced. Violations ⇒ `C_ATT_MISSING`, `C_ORD_DRIFT`.
Non-determinism (A2)	EnvLock/ReplayPlan, determinism probes (§4.6)	Pinned FP/INT modes, seeds, codecs. Replay mismatch in incentives ⇒ `C_INC_NONDET`; otherwise `C_LEDGER_DRIFT`/`C_ORD_DRIFT`.
Ledger manipulation (A3)	`CBF_LEDGER_INTEGRITY` receipt (§??)	Duplicate (`edge_id`, `sequence_no`) must re-execute identically; conservation across window required. Violations ⇒ `C_LEDGER_DRIFT`.
Governance capture (A4)	UpdateCert + `GOV_UPDATE_APPLIED` (§??)	Exclusive update path, quorum check, bisimulation + Σ-non-expansion. Fail ⇒ `C_GOV_NO_CERT`.
Hash/digest ambiguity (A5)	BLAKE3-only (N0), LegacyRebindCert (§3)	Mixed hashes prohibited for new artifacts; legacy must be dual-bound. Missing rebinding ⇒ `C_ORD_DRIFT`.
Axis bleed / unknown laundering (A6)	Axis isolation (N4), Guard-first (N3)	Undeclared writes ⇒ `C_AXIS_LEAK`; priced ⊥ at guard ⇒ `C_UNK_PRICED`.
Incentive feedback / bias (A7)	No in-window feedback; budget at CBF (§??)	Guards cannot read incentive axes in-window; budget failure ⇒ `C_INC_BUDGET`.
Sandbox escape / ethics (A8)	NonActuationSandboxCert, `SANDBOX_SAFE`	Only `HypoImpact` writes permitted; ethics violations ⇒ `C_ETH_GUARD`, priced write ⇒ `C_SANDBOX_WRITE`.

5.3 Residual Risks and Operator Guidance

- **Policy misconfiguration.** Incorrect thresholds/budgets can still make acceptance too lax or too strict. Mitigate with change control via UpdateCert, rate limits/hysteresis, and staged rollouts.
- **Side-channeling via metadata.** Clamp labels and metadata are excluded from acceptance but may leak signals. Treat metadata as public; avoid sensitive content in free-form fields.
- **Denial via obligation flood.** Excess creation of low-priority obligations can delay progress. Rank μ and non-expansion (O1–O3) constrain but do not replace capacity planning.

5.4 Operational Recommendations

- **CI gate on conformance** (§4): horizon tests, manifest reconstruction, CBF audit, receipt schema validation.
- **Pin kernels by `checker_hash`** in deployment manifests; reject unpinned bytes.
- **Roll updates behind UpdateCert** with automated bisimulation harnesses that replay recent `proof_surface` samples and check ACCEPT/HALT extensionality.
- **Log RAW elevations** with RAWGrant and expiries; audit periodically for drift.

Security stance. Part 5's guarantees are *evidence-first*: every mitigation has a corresponding receipt and digest included in the Compliance Bundle, enabling third parties to verify security properties by replay rather than trust.

6 Worked End-to-End Examples

This section presents two minimal, end-to-end windows: a *PASS* run and a *HALT* run. Each shows the guard receipts, the Ω horizon gate, the ReplayReceipt, and (for HALT) the RejectionCert. Digests are placeholders.

6.1 Example A — PASS Window

Setup. Policy pins budgets B so that all priced coordinates are within limits; ethics and rounding caps are satisfied. Window parameters: τ=256, $\tau_{\min}$=64, N=100 $\Rightarrow$ $K = \max(100, \lceil 512/64 \rceil = 8) = 100$.

Listing 9: PASS: Compliance Bundle (abridged)

Compliance Bundle (abridged).

```
{
  "attestation_receipt": "b3:att-aa11",
  "guard_receipts": ["b3:ef-bb22","b3:dro-cc33","b3:pr-dd44","b3:cbf-ee55"
     ],
  "omega_receipt": "b3:om-ff66",
  "bundle_manifest": "b3:mr-gg77",
  "axis_catalog_digest": "b3:ax-hh88",
  "policy_digests": ["b3:bud-ii99","b3:eth-jj00","b3:round-kk11"]
}
```

Listing 10: EF guard receipt (PASS)

EF receipt.

```
{
  "predicate_id": "EF_LOCALITY",
  "inputs_digest": "b3:x-01",
  "checker_hash": "b3:chk-ef",
  "outcome": { "verdict": "PASS" },
  "clamp": "GREEN",
  "time": 5, "stage": "EF"
}
```

Listing 11: DRO guard receipt (PASS)

DRO receipt.

```
{
  "predicate_id": "DRO_SIDECHANNEL",
  "inputs_digest": "b3:x-02",
  "checker_hash": "b3:chk-dro",
  "outcome": { "verdict": "PASS" },
  "time": 12, "stage": "DRO"
}
```

Listing 12: PR receipt (PASS; no retune needed)

PR receipt.

```
{
  "predicate_id": "PR_PROFILE",
  "inputs_digest": "b3:x-03",
  "checker_hash": "b3:chk-pr",
  "outcome": { "verdict": "PASS" },
  "time": 40, "stage": "PR"
}
```

CBF receipts. One packet crosses a boundary and debits an incentive axis within budget; audit passes.

Listing 13: CrossingPacket + CBF$_AUDIT(PASS)$

```
{
  "packet": {
    "edge_id": "cbf:v1:edge-A",
    "inputs_digest": "b3:inp-a1",
    "kernel_id@hash": "inc_kernel@b3:k1",
    "priced_delta": { "AuditPool": -10.0, "Risk": 0.0 },
    "sequence_no": 1,
    "prev_hash": "b3:zero",
    "packet_hash": "b3:ph-11"
  },
  "cbf_audit": {
    "predicate_id": "CBF_LEDGER_INTEGRITY",
    "packet_sequence_digest": "b3:ps-22",
    "iss_initial_state_digest": "b3:is-33",
    "iss_final_state_digest": "b3:ie-44",
    "checker_hash": "b3:chk-cbf",
    "outcome": { "verdict": "PASS" },
    "time": 120, "stage": "CBF"
  }
}
```

Listing 14: OmegaReceipt (PASS) and ReplayReceipt

OmegaReceipt and ReplayReceipt.

```
{
  "omega": {
    "predicate_id": "OMEGA_HORIZON",
```

```
    "inputs_digest": "b3:tuple-  Nmin  ",
    "checker_hash": "b3:chk-omega",
    "computed_k": 100,
    "outcome": { "verdict": "PASS" },
    "time": 200, "stage": "Omega"
  },
  "replay": {
    "kind": "ReplayReceipt",
    "bundle_manifest": "b3:mr-gg77",
    "proof_surface": "b3:surf-pass-EXA",
    "envlock_digest": "b3:env-01",
    "replayplan_digest": "b3:rp-01",
    "axis_catalog_digest": "b3:ax-hh88",
    "outcome": { "verdict": "ACCEPT" },
    "time": 205
  }
}
```

6.2 Example B — HALT on Horizon Gate

Setup. Window parameters: τ=100, $\tau_{\min}$=60, N=10. Horizon precondition fails since $100 < 2 \cdot 60$.

Listing 15: EF/DRO/PR/CBF PASS; Omega FAIL

Upstream guards (PASS), terminal FAIL.

```
{
  "guard_receipts": ["b3:ef-ok","b3:dro-ok","b3:pr-ok","b3:cbf-ok"],
  "omega": {
    "predicate_id": "OMEGA_HORIZON",
    "inputs_digest": "b3:tuple-  Nmin  ",
    "checker_hash": "b3:chk-omega",
    "computed_k": 20,                    // producers computation
    "outcome": { "verdict": "FAIL", "reason_code": "C_HZN_VIOL" },
    "time": 140, "stage": "Omega"
  }
}
```

Listing 16: RejectionCert at Ω

RejectionCert (minimal counterexample).

```
{
  "kind": "RejectionCert",
  "predicate_id": "OMEGA_HORIZON",
  "counterexample": {
    "inputs_digest": "b3:tuple-  Nmin  ",
    "witness": { "tau": 100, "tau_min": 60, "N": 10 },
    "reason_code": "C_HZN_VIOL"
  },
  "time": 141,
  "stage": "Omega"
}
```

Listing 17: ReplayReceipt (HALT)

Terminal ReplayReceipt (HALT).

```
{
  "kind": "ReplayReceipt",
  "bundle_manifest": "b3:mr-fail",
  "proof_surface": "b3:surf-halt-EXB",
  "envlock_digest": "b3:env-01",
  "replayplan_digest": "b3:rp-01",
  "axis_catalog_digest": "b3:ax-hh88",
  "outcome": { "verdict": "HALT" },
  "time": 145
}
```

6.3 Notes

- In Example A, incentive spending is accounted at CBF, conservation holds, and guards do not read incentive axes within the window (no-feedback).
- In Example B, upstream passes do not override the horizon rule; the Loader halts with `C_HZN_VIOL` and emits a minimal counterexample.
- Both examples bind all receipts and packets into the manifest and thus into `proof_surface`, enabling byte-identical replay.

7 Versioning, Registry Stability, and Change Log

This section fixes how Part 5 evolves without breaking replay-equivalence or validator compatibility. It defines version markers, stability guarantees, and the deprecation playbook that ties into migration (§3).

7.1 Version Markers and Compatibility

V0. Canon/ABI versioning.

- **canon/v1** — Canonical JSON codec used throughout Part 5 (sorted keys, minimal decimals, UTF-8). Changing codec rules *bumps canon* and requires a gated UpdateCert with bisimulation proof for all archived bundles.
- **ABI-v1** — Predicate registry and receipt field minima (§??, §4.1). Adding new *optional* fields preserves ABI; changing or removing *required* fields requires an ABI bump.

Semantic versioning. We publish `part5` version as `MAJOR.MINOR.PATCH`:

- **MAJOR**: Incompatible changes (e.g., new canon, receipt minima change).
- **MINOR**: Backward-compatible additions (new predicates, optional keys).
- **PATCH**: Editorial fixes and clarifications (no schema/logic change).

Predicate registry stability. IDs in Table 2 are *stable*. New predicates *append* to the registry; existing IDs are never repurposed. A predicate's `checker_hash` may change only via UpdateCert, and old runs remain replayable because receipts pin the historical `checker_hash`.

7.2 Deprecation and Sunsetting

V1. Deprecation lifecycle.

1. **Announce:** Publish a deprecation notice in policy (digest appears in `policy_digests`); provide a migration path and test vectors.
2. **Coexist:** Allow old + new forms concurrently with dual-binding (if relevant). Validators emit *warnings* but still PASS.
3. **Enforce:** Flip to *errors* via UpdateCert; validators must reject deprecated-only artifacts (codes per §??).

Non-breaking additions. Adding: new optional receipt keys, new unpriced diagnostic axes (e.g., more `HypoImpact` coordinates), or new failure codes is non-breaking; CI must tolerate unknown optional keys.

Breaking changes (examples). Changing required fields in the uniform receipt, altering Merkle leaf ordering, or modifying the horizon function semantics are *breaking* and require: (i) MAJOR bump; (ii) UpdateCert with bisimulation proof; (iii) migration instructions (§3).

7.3 Change Log (since canon/v1, ABI-v1)

Version	Date	Notes
`1.0.0`	2025-09-17	Initial normative release of Part 5 on **canon/v1**, **ABI-v1**. Single hash primitive (BLAKE3), Compliance Bundle, Omega horizon gate (K), Crossing-Packet + CBF invariants, certified governance via UpdateCert, deterministic incentives, sandbox isolation to `HypoImpact`, migration with LegacyRebindCert, conformance suite.
`1.0.1`	2025-09-17	Editorial: fixed registry table captions and clarified `INCENTIVE_DEBIT` as CBF-stage predicate; no schema change.

7.4 Validator Expectations Across Versions

V2. Cross-version rules.

- **Strict on canon/ABI:** A validator declares the highest `canon/ABI` it supports; it must accept any bundle at or below that level and reject higher levels with a clear reason.
- **Lenient on optional keys:** Unknown optional keys *must not* cause failure; ignore them during validation.
- **Legacy acceptance:** Acceptance of legacy artifacts requires presence of LegacyRebindCert and dual-binding per §3.3.
- **Predicate extension:** Unknown `predicate_id`s are allowed if their receipts validate

structurally; downstream policy can decide to ignore.

- **Checker pinning:** For historical runs, re-run kernels identified by their historical `checker_hash`; do not substitute newer bytes.

7.5 Release Artifacts

Each Part 5 release publishes:

- A machine-readable registry JSON (predicate IDs, required fields, stages).
- Reference codecs and manifest calculators (hashes of binaries + seeds).
- Conformance test bundles and expected `proof_surface` values.
- Migration scripts for legacy to `proof_surface` with sample LegacyRebindCert.

Stability guarantee. Within a MAJOR version, existing bundles remain valid and replayable. New MINOR/PATCH releases cannot change required fields, the horizon gate semantics, Merkle ordering, or the single-hash discipline.